PERCOBAAN 7
DECODER 7 SEGMENT

NAMA: Ihsanta Zaki Sanjaya

KELAS: 1 D4 Teknik Komputer B

NRP: 3223600058

DOSEN: Reni Soelistijorini B.Eng, MT

MATA KULIAH: Praktikum Rangkaian Logika 2

TANGGAL: Kamis, 23 Mei 2024

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2024

Tujuan:

1. Mampu mengimplementasikan algoritma decoder 7 segmen.
2. Dapat menampilkan nilai masukan keluaran pada 7 segmen dalam modul FPGA.

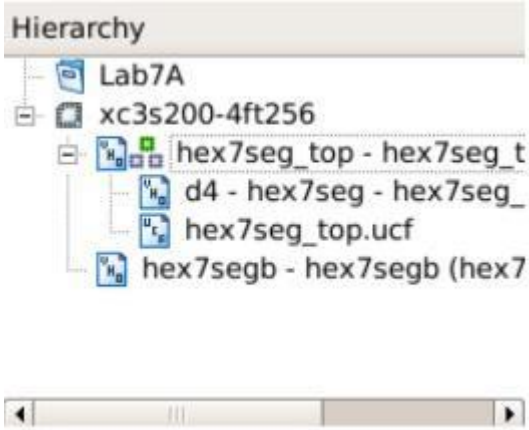
Alat dan Bahan:

1. PC beserta software ISE Design Suite 13.1
2. Xilinx Spartan 3
3. Downloader JTAG USB
4. Adaptor 5V

Percobaan A.7-

Segmen Display

Membuat Project dengan nama Lab7A



A. Program hex7seg.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity hex7seg is
Port (x : in STD_LOGIC_VECTOR(3 downto 0); a_to_g
: out STD_LOGIC_VECTOR(6 downto 0));

end hex7seg;
architecture hex7seg_le of hex7seg is begin
a_to_g(6) <= (not x(3) and not x(2) and not x(1) and x(0)) or (not
x(3) and x(2) and not x(1) and not x(0)) or
(x(3) and x(2) and not x(1) and x(0)) or
(x(3) and not x(2) and x(1) and x(0));
```

```
a_to_g(5) <= (x(2) and x(1) and not x(0)) or
(x(3) and x(1) and x(0)) or
(not x(3) and x(2) and not x(1) and x(0)) or
(x(3) and x(2) and not x(1) and not x(0));
a_to_g(4) <= (not x(3) and not x(2) and x(1) and not x(0)) or

(x(3) and x(2) and x(1)) or
(x(3) and x(2) and not x(0));

a_to_g(3) <= (not x(3) and not x(2) and not x(1) and x(0)) or (not
x(3) and x(2) and not x(1) and not x(0)) or
(x(3) and not x(2) and x(1) and not x(0)) or
(x(2) and x(1) and x(0));
a_to_g(2) <= (not x(3) and x(0)) or
(not x(3) and x(2) and not x(1)) or
(not x(2) and not x(1) and x(0));
a_to_g(1) <= (not x(3) and not x(2) and x(0)) or (not
x(3) and not x(2) and x(1)) or
(not x(3) and x(1) and x(0)) or
(x(3) and x(2) and not x(1) and x(0));
a_to_g(0) <= (not x(3) and not x(2) and not x(1)) or
(x(3) and x(2) and not x(1) and not x(0)) or
(not x(3) and x(2) and x(1) and x(0));

end hex7seg_le;
```

B. Program hex7segb.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity hex7segb is
port (
x : in STD_LOGIC_VECTOR(3 downto 0); a_to_g :
out STD_LOGIC_VECTOR(6 downto 0)
);
end hex7segb;
architecture hex7segb of hex7segb is begin
```

```
process(x)
begin  case
x is
when x"0" => a_to_g <= "0000001"; -- 0
when x"1" => a_to_g <= "1001111"; -- 1
when x"2" => a_to_g <= "0010010"; -- 2
when x"3" => a_to_g <= "0000110"; -- 3
when x"4" => a_to_g <= "1001100"; -- 4
when x"5" => a_to_g <= "0100100"; -- 5
when x"6" => a_to_g <= "0100000"; -- 6
when x"7" => a_to_g <= "0001111"; -- 7
when x"8" => a_to_g <= "0000000"; -- 8
when x"9" => a_to_g <= "0000100"; -- 9
when x"A" => a_to_g <= "0001000"; -- A
when x"B" => a_to_g <= "1100000"; -- B
when x"C" => a_to_g <= "0110001"; -- C
when x"D" => a_to_g <= "1000010"; -- D
when x"E" => a_to_g <= "0110000"; -- E
when others => a_to_g <= "0111000"; -- F
end case;
end process; end
hex7segb;
```

C. Program hex7seg_top.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity hex7seg_top is
Port (
sw : in STD_LOGIC_VECTOR(3 downto 0); a_to_g :
out STD_LOGIC_VECTOR(6 downto 0); an : out
STD_LOGIC_VECTOR(3 downto 0);
dp : out STD_LOGIC
);
end hex7seg_top;
architecture hex7seg_top of hex7seg_top is
component hex7seg is
port(
```

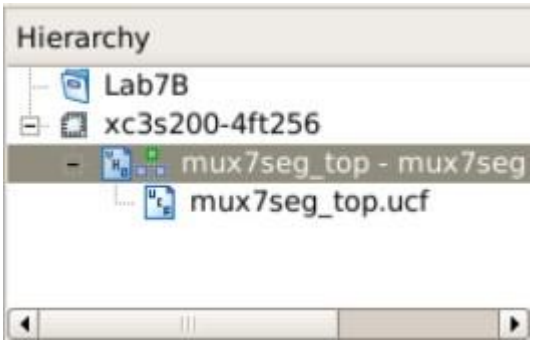
```
x: in STD_LOGIC_VECTOR(3 downto 0);  a_to_g :
out STD_LOGIC_VECTOR(6 downto 0)
);
end component;
begin
an <= "0000"; -- all digit on
dp <= '1'; -- dp off
d4: hex7seg port map (x => sw, a_to_g => a_to_g); end
hex7seg_top;
```

D. Program hex7seg_top.ucf (konfigurasi pin)

```
NET "a_to_g[0]" LOC = N16;
NET "a_to_g[1]" LOC = F13;
NET "a_to_g[2]" LOC = R16;
NET "a_to_g[3]" LOC = P15;
NET "a_to_g[4]" LOC = N15;
NET "a_to_g[5]" LOC = G13;
NET "a_to_g[6]" LOC = E14;
NET "an[0]" LOC = D14; NET
"an[1]" LOC = G14; NET
"an[2]" LOC = F14;  NET
"an[3]" LOC = E13;  NET "dp"
LOC = F12;
NET  "sw[0]"  LOC  =  F12;
NET  "sw[1]"  LOC  =  G12;
NET  "sw[2]"  LOC  =  H14;
NET "sw[3]" LOC = H13;
```

E. Ouput

B. Multiplexing 7-Segmen



a. Program mux7seg_top.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity mux7seg_top is Port
(
btn : in STD_LOGIC_VECTOR(3 downto 0); a_to_g :
out STD_LOGIC_VECTOR(6 downto 0); an : out
STD_LOGIC_VECTOR(3 downto 0);
dp : out STD_LOGIC
);
end mux7seg_top;
architecture mux7seg of mux7seg_top is  signal
x: STD_LOGIC_VECTOR(15 downto 0); signal s:
STD_LOGIC_VECTOR(1 downto 0);
signal digit: STD_LOGIC_VECTOR(3 downto 0); begin
x <= x"0058";
an <= not btn;
s(1) <= btn(2) or btn(3);
s(0) <= btn(1) or btn(3);
dp <= '1';
-- quad 4 to 1 mux: mux44
process(s)
begin case
s is
when "00" => digit <= x(3 downto 0);
when "01" => digit <= x(7 downto 4);
when "10" => digit <= x(11 downto 8);
```

```
when others => digit <= x(15 downto 12);
end case;
end process;
process(digit)
begin
case digit is
when x"0" => a_to_g <= "0000001"; -- 0
when x"1" => a_to_g <= "1001111"; -- 1
when x"2" => a_to_g <= "0010010"; -- 2
when x"3" => a_to_g <= "0000110"; -- 3
when x"4" => a_to_g <= "1001100"; -- 4
when x"5" => a_to_g <= "0100100"; -- 5
when x"6" => a_to_g <= "0100000"; -- 6
when x"7" => a_to_g <= "0001111"; -- 7
when x"8" => a_to_g <= "0000000"; -- 8
when x"9" => a_to_g <= "0000100"; -- 9
when x"A" => a_to_g <= "0001000"; -- A
when x"B" => a_to_g <= "1100000"; -- B
when x"C" => a_to_g <= "0110001"; -- C
when x"D" => a_to_g <= "1000010"; -- D
when x"E" => a_to_g <= "0110000"; -- E
when others => a_to_g <= "0111000"; -- F
end case;
end process; end
mux7seg;
```

b. Program mux7seg_top.ucf (konfigurasi pin)

```
NET "a_to_g[0]" LOC = N16;
NET "a_to_g[1]" LOC = F13;
NET "a_to_g[2]" LOC = R16;
NET "a_to_g[3]" LOC = P15;
NET "a_to_g[4]" LOC = N15;
NET "a_to_g[5]" LOC = G13;
NET "a_to_g[6]" LOC = E14;
NET "an[0]" LOC = D14;
NET "an[1]" LOC = G14;
```

NET "an[2]" LOC = F14;

NET "an[3]" LOC = E13;

NET "dp" LOC = F12;

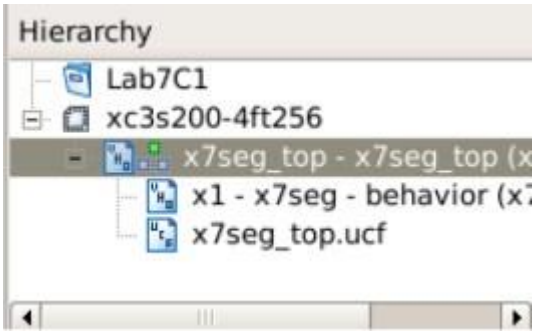
NET "btn[0]" LOC = M13;

NET "btn[1]" LOC = M14;

NET "btn[2]" LOC = L13;

NET "btn[3]" LOC = L14;

C. Ouput



C. Multiplexing 7segmen Display dengan ClockDevider (bagian 1)

a. Program x7seg.vhdl

library IEEE;

```
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL; entity
x7seg is
Port (
```

```
x : in STD_LOGIC_VECTOR(15 downto 0); clk :
in STD_LOGIC;
clr : in STD_LOGIC;
a_to_g : out STD_LOGIC_VECTOR(6 downto 0); an :
out STD_LOGIC_VECTOR(3 downto 0);
dp : out STD_LOGIC
);
end x7seg;
architecture x7seg of x7seg is
signal s : STD_LOGIC_VECTOR(1 downto 0); signal
digit : STD_LOGIC_VECTOR(3 downto 0); signal aen :
STD_LOGIC_VECTOR(3 downto 0); signal clkdiv :
STD_LOGIC_VECTOR(20 downto 0); begin
s <= clkdiv(20 downto 19);
aen <= "1111";
dp <= '1';
-- Quad 4 to 1 mux
process(s, x)  begin
case s is
when "00" => digit <= x(3 downto 0);
when "01" => digit <= x(7 downto 4);
when "10" => digit <= x(11 downto 8);
when others => digit <= x(15 downto 12);
end case;
end process;
-- 7-segment decoder: hex7seg
process(digit)
begin
case digit is
when x"0" => a_to_g <= "0000001"; -- 0
when x"1" => a_to_g <= "1001111"; -- 1
when x"2" => a_to_g <= "0010010"; -- 2
when x"3" => a_to_g <= "0000110"; -- 3
when x"4" => a_to_g <= "1001100"; -- 4
when x"5" => a_to_g <= "0100100"; -- 5
```

```
when x"6" => a_to_g <= "0100000"; -- 6
when x"7" => a_to_g <= "0001111"; -- 7
when x"8" => a_to_g <= "0000000"; -- 8
when x"9" => a_to_g <= "0000100"; -- 9
when x"A" => a_to_g <= "0001000"; -- A
when x"B" => a_to_g <= "1100000"; -- B
when x"C" => a_to_g <= "0110001"; -- C
when x"D" => a_to_g <= "1000010"; -- D
when x"E" => a_to_g <= "0110000"; -- E
when others => a_to_g <= "0111000"; -- F
end case;
end process;
-- Digit select: ancode
process(s, aen)
begin
an <= "1111";
if aen(conv_integer(s)) = '1' then
an(conv_integer(s)) <= '0';
end if;
end process;
-- Clock divider
process(clk, clr)
begin
if clr = '1' then
clkdiv <= (others => '0');
elsif clk'event and clk = '1' then
clkdiv <= clkdiv + 1;
end if;
end process; end
x7seg;
```

b. Program x7seg_top.vhdl

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity x7seg_top is
Port (
```

```
mclk : in STD_LOGIC; btn :
in STD_LOGIC;
a_to_g : out STD_LOGIC_VECTOR(6 downto 0); an :
out STD_LOGIC_VECTOR(3 downto 0);
dp : out STD_LOGIC
);
end x7seg_top;
architecture x7seg_top of x7seg_top is
component x7seg is
Port (
x : in STD_LOGIC_VECTOR(15 downto 0); clk :
in STD_LOGIC;
clr : in STD_LOGIC;
a_to_g : out STD_LOGIC_VECTOR(6 downto 0);

an : out STD_LOGIC_VECTOR(3 downto 0); dp :
out STD_LOGIC
);
end component;
signal x: STD_LOGIC_VECTOR(15 downto 0); begin
x <= x"B058";
x1 : x7seg port map ( x
=> x,
clk => mclk,
clr => btn,
a_to_g => a_to_g, an
=> an,
dp => dp
);
end x7seg_top;
```

c. Program x7seg_top.ucf (konfigurasi pin)

```
NET "a_to_g[0]" LOC = N16;
NET "a_to_g[1]" LOC = F13;
NET "a_to_g[2]" LOC = R16;
```

NET "a_to_g[3]" LOC = P15;

NET "a_to_g[4]" LOC = N15;

NET "a_to_g[5]" LOC = G13;

NET "a_to_g[6]" LOC = E14;

NET "an[0]" LOC = D14; NET

"an[1]" LOC = G14; NET

"an[2]" LOC = F14; NET

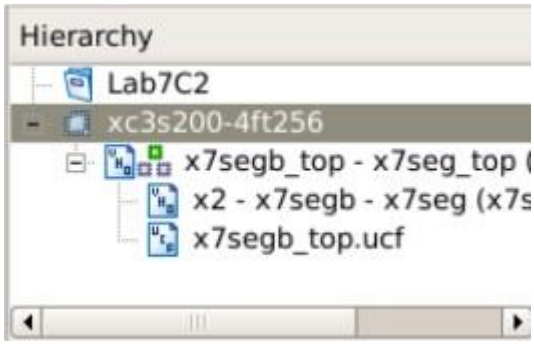
"an[3]" LOC = E13; NET "dp"

LOC = P16;

NET "btn" LOC = M13;

C. Output

C. Multiplexing 7segmen Display dengan ClockDevider (bagian 2)



a. Program x7segb.vhdl

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.STD_LOGIC_unsigned.ALL;

```
entity x7segb is
Port (
x : in STD_LOGIC_VECTOR(15 downto 0); clk :
in STD_LOGIC;
clr : in STD_LOGIC;
a_to_g : out STD_LOGIC_VECTOR(6 downto 0); an :
out STD_LOGIC_VECTOR(3 downto 0);
dp : out STD_LOGIC
);
end x7segb;
architecture x7seg of x7segb is
signal s : STD_LOGIC_VECTOR(1 downto 0); signal
digit : STD_LOGIC_VECTOR(3 downto 0); signal aen :
STD_LOGIC_VECTOR(3 downto 0); signal clkdiv :
STD_LOGIC_VECTOR(20 downto 0); begin
s <= clkdiv(20 downto 19);
dp <= '1';
-- Set aen(3 downto 0) for leading blanks
aen(3) <= x(15) or x(14) or x(13) or x(12);
aen(2) <= x(15) or x(14) or x(13) or x(12) or x(11) or x(10) or
x(9) or x(8);
aen(1) <= x(15) or x(14) or x(13) or x(12) or x(11) or x(10) or
x(9) or x(8) or x(7) or x(6) or x(5) or x(4);
aen(0) <= '1'; -- digit 0 always on
-- Quad 4 to 1 mux
process(s, x)  begin
case s is
when "00" => digit <= x(3 downto 0);

when "01" => digit <= x(7 downto 4);
when "10" => digit <= x(11 downto 8);
when others => digit <= x(15 downto 12);
end case;
end process;
-- 7-segment decoder: hex7seg
```

```
process(digit)
begin
case digit is
when x"0" => a_to_g <= "0000001"; -- 0
when x"1" => a_to_g <= "1001111"; -- 1
when x"2" => a_to_g <= "0010010"; -- 2
when x"3" => a_to_g <= "0000110"; -- 3
when x"4" => a_to_g <= "1001100"; -- 4
when x"5" => a_to_g <= "0100100"; -- 5
when x"6" => a_to_g <= "0100000"; -- 6
when x"7" => a_to_g <= "0001111"; -- 7
when x"8" => a_to_g <= "0000000"; -- 8
when x"9" => a_to_g <= "0000100"; -- 9
when x"A" => a_to_g <= "0001000"; -- A
when x"B" => a_to_g <= "1100000"; -- B
when x"C" => a_to_g <= "0110001"; -- C
when x"D" => a_to_g <= "1000010"; -- D
when x"E" => a_to_g <= "0110000"; -- E
when others => a_to_g <= "0111000"; -- F
end case;
end process;
-- Digit select: ancode
process(s, aen)
begin
an <= "1111";
if aen(conv_integer(s)) = '1' then
an(conv_integer(s)) <= '0';
end if;
end process;
--  Clock  Divider
process(clk,  clr)
begin
if clr = '1' then
clkdiv <= (others => '0');
elsif clk'event and clk = '1' then
clkdiv <= clkdiv + 1;
end if;
```

```
end process; end
x7seg;
```

b. Program x7segb_top.vhdl

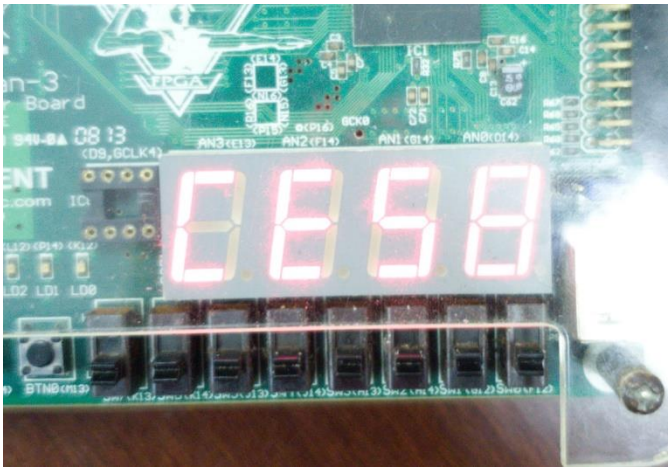
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity x7segb_top is
Port (
clk : in STD_LOGIC;
btn : in STD_LOGIC_VECTOR (3 downto 0); sw :
in STD_LOGIC_VECTOR (7 downto 0);
a_to_g : out STD_LOGIC_VECTOR (6 downto 0); an :
out STD_LOGIC_VECTOR (3 downto 0);

dp : out STD_LOGIC
);
end x7segb_top;
architecture x7seg_top of x7segb_top is component
x7segb is
Port (
x : in STD_LOGIC_VECTOR (15 downto 0); clk :
in STD_LOGIC;
clr : in STD_LOGIC;
a_to_g : out STD_LOGIC_VECTOR (6 downto 0); an :
out STD_LOGIC_VECTOR (3 downto 0);
dp : out STD_LOGIC
);
end component;
signal x : STD_LOGIC_VECTOR (15 downto 0); begin
-- concatenate switch and 3 buttons
x <= sw & btn(2 downto 0) & "0000100"; -- digit 0 =
2 x2 : x7segb port map (
x => x,
clk => clk,
clr => btn(3),
```

```
a_to_g => a_to_g, an
=> an,
dp => dp
);
end x7seg_top;
```

c. Program x7seg_top.ucf (konfigurasi pin)

```
NET "a_to_g[0]" LOC = N16;
NET "a_to_g[1]" LOC = F13;
NET "a_to_g[2]" LOC = R16;
NET "a_to_g[3]" LOC = P15;
NET "a_to_g[4]" LOC = N15;
NET "a_to_g[5]" LOC = G13;
NET "a_to_g[6]" LOC = E14;
NET "an[0]" LOC = D14; NET
"an[1]" LOC = G14; NET
"an[2]" LOC = F14;  NET
"an[3]" LOC = E13;  NET "dp"
LOC = P16;
NET "btn[0]" LOC = M13;
NET "btn[1]" LOC = M14;
NET "btn[2]" LOC = L13; NET
"btn[3]" LOC = L14; NET
"sw[0]" LOC = F12; NET
"sw[1]" LOC = G12; NET
"sw[2]" LOC = H14; NET
"sw[3]" LOC = H13; NET
"sw[4]" LOC = J14; NET
"sw[5]" LOC = J13; NET
"sw[6]" LOC = K14; NET
"sw[7]" LOC = K13;
```

D. Output



ANALISA

- Program hex7seg

Entitas `hex7seg` dan `hex7segb` mendefinisikan dua metode berbeda untuk mengontrol segmen 7-segmen: `hex7seg` menggunakan logika kombinasi untuk mengaktifkan segmen, sementara `hex7segb` menggunakan pernyataan `case` dalam sebuah proses untuk mencapai hal yang sama. Entitas tingkat atas `hex7seg_top` menghubungkan input switch (sw) dengan output segmen (a_to_g) melalui komponen `hex7seg`, mengaktifkan semua digit (an <= "0000"), dan mematikan titik desimal (dp <= '1'). Bagian file constraint (NET ... LOC = ...) mengaitkan sinyal logika ini dengan pin fisik spesifik pada FPGA yang digunakan. Program ini memungkinkan tampilan 7-segmen untuk menampilkan angka 0-F berdasarkan posisi switch input.

- Multiplexer 7 Segmen

Program `mux7seg_top` mengimplementasikan pengontrol tampilan 7-segmen menggunakan multiplexer 4:1 yang memanfaatkan tombol sebagai input untuk memilih digit yang akan ditampilkan. Sinyal `x` diinisialisasi dengan nilai heksadesimal 0058, yang berisi digit heksadesimal 0, 5, dan 8. Tombol (btn) digunakan untuk mengontrol sinyal `s`, yang kemudian menentukan digit mana yang akan ditampilkan pada tampilan 7-segmen melalui proses multiplexer. Sinyal `an` diinisialisasi dengan nilai kebalikan dari `btn` untuk mengontrol anoda (mengaktifkan digit tertentu), dan `dp` diatur ke mati ('1'). Proses pertama menggunakan pernyataan `case` untuk memilih digit dari sinyal `x` berdasarkan nilai `s`, sementara proses kedua menerjemahkan digit tersebut menjadi pola segmen yang sesuai untuk ditampilkan pada tampilan 7-segmen (a_to_g). File UCF mengkonfigurasi pin fisik pada FPGA, menghubungkan sinyal internal ke pin fisik yang sesuai. Program ini memungkinkan tampilan 7-segmen menampilkan salah satu dari empat digit berdasarkan tombol yang ditekan.

- Multiplexing 7-Segmen Display dengan Clock Divider (bagian 1)

Program `x7seg` mengimplementasikan pengontrol tampilan 7-segmen menggunakan teknik multiplexing dan pembagi frekuensi clock. Komponen ini menerima input 16-bit (x), clock (clk), dan sinyal reset (clr). Sinyal clkdiv digunakan sebagai pembagi frekuensi untuk menghasilkan sinyal pemilihan digit (s). Empat digit input (x) dipilih berdasarkan nilai s melalui proses multiplexer, dan masing-masing digit diterjemahkan ke dalam pola segmen 7-segmen (a_to_g) menggunakan decoder. Anoda tampilan (an) dikendalikan oleh sinyal enable (aen), dan titik desimal (dp) dimatikan ('1'). Komponen ini terhubung dalam entitas x7seg_top, yang menghubungkan sinyal global clock (mclk) dan tombol reset (btn) ke sinyal komponen x7seg. File constraint (x7seg_top.ucf) mengkonfigurasi pin fisik pada FPGA untuk menghubungkan sinyal logika dengan perangkat keras yang sesuai. Program ini memungkinkan tampilan 7-segmen untuk menampilkan empat digit dari input 16-bit dengan menggunakan teknik multiplexing dan kontrol clock.

- Multiplexing 7-Segmen Display dengan Clock Divider (bagian 2)

Program `x7segb` juga mengimplementasikan pengontrol tampilan 7-segmen menggunakan teknik multiplexing dan pembagi frekuensi clock. Entitas ini menerima input 16-bit (x), clock (clk), dan sinyal reset (clr). Sinyal clkdiv berfungsi sebagai pembagi frekuensi untuk menghasilkan sinyal pemilihan digit (s). Digit yang dipilih dari input x berdasarkan nilai s diterjemahkan ke dalam pola segmen 7-segmen (a_to_g) menggunakan proses decoder. Sinyal enable (aen) mengontrol anoda (an), memungkinkan tampilan digit yang sesuai, dan titik desimal (dp) dimatikan ('1'). Pada entitas tingkat atas x7segb_top, sinyal global clock (clk) dan tombol reset (btn(3)) digunakan untuk mengontrol komponen x7segb. Sinyal input 16-bit (x) dibentuk dengan menggabungkan switch (sw) dan tiga tombol (btn(2:0)), dengan digit terakhir diatur ke 9 (nilai biner "0000100"). File constraint (x7seg_top.ucf) mengonfigurasi pin fisik pada FPGA, menghubungkan sinyal logika ke perangkat keras yang sesuai. Program ini memungkinkan tampilan 7-segmen untuk menampilkan empat digit dari input 16-bit dengan menggunakan teknik multiplexing dan kontrol clock, serta menyediakan kemampuan untuk mengatur tampilan melalui switch dan tombol.

D. Mini Project

A. Program miniproject.vhdl

```
entity miniproject is
port(
sw : in STD_LOGIC_VECTOR (3 downto 0);
clock_100Mhz : in STD_LOGIC;
reset : in STD_LOGIC;
an : out STD_LOGIC_VECTOR (3 downto 0);
dp : out STD_LOGIC;
oo : out STD_LOGIC_VECTOR (6 downto 0)
);
end miniproject;
architecture Behavioral of miniproject is
signal refresh: STD_LOGIC_VECTOR (19 downto 0);
signal tambah: std_logic_vector(1 downto 0);
signal a_to_g : STD_LOGIC_VECTOR (6 downto 0);
signal a_to_h : STD_LOGIC_VECTOR (6 downto 0);
signal a_to_i : STD_LOGIC_VECTOR (6 downto 0);
signal a_to_j : STD_LOGIC_VECTOR (6 downto 0);
begin
dp <= '1';
process(sw,a_to_g,a_to_h,a_to_i,a_to_j)
begin
case sw is
when "0000" => a_to_g <= "0000001" ; a_to_h <= "0000001" ; a_to_i <= "1001111" ; a_to_j <= "0000001";
when "0001" => a_to_g <= "1001111" ; a_to_h <= "0000001" ; a_to_i <= "0010010" ; a_to_j <= "0000001";
when "0010" => a_to_g <= "0010010" ; a_to_h <= "0000001" ; a_to_i <= "0000110" ; a_to_j <= "0000001";
when "0011" => a_to_g <= "0000110" ; a_to_h <= "0000001" ; a_to_i <= "1001100" ; a_to_j <= "0000001";
when "0100" => a_to_g <= "1001100" ; a_to_h <= "0000001" ; a_to_i <= "0100100" ; a_to_j <= "0000001";
when "0101" => a_to_g <= "0100100" ; a_to_h <= "0000001" ; a_to_i <= "0100000" ; a_to_j <= "0000001";
when "0110" => a_to_g <= "0100000" ; a_to_h <= "0000001" ; a_to_i <= "0001111" ; a_to_j <= "0000001";
when "0111" => a_to_g <= "0001111" ; a_to_h <= "0000001" ; a_to_i <= "0000000" ; a_to_j <= "0000001";
when "1000" => a_to_g <= "0000000" ; a_to_h <= "0000001" ; a_to_i <= "0000100" ; a_to_j <= "0000001";
when "1001" => a_to_g <= "0000100" ; a_to_h <= "0000001" ; a_to_i <= "0000001" ; a_to_j <= "1001111";
when "1010" => a_to_g <= "0000001" ; a_to_h <= "1001111" ; a_to_i <= "1001111" ; a_to_j <= "1001111";
when "1011" => a_to_g <= "1001111" ; a_to_h <= "1001111" ; a_to_i <= "0010010" ; a_to_j <= "1001111";
when "1100" => a_to_g <= "0010010" ; a_to_h <= "1001111" ; a_to_i <= "0000110" ; a_to_j <= "1001111";
when "1101" => a_to_g <= "0000110" ; a_to_h <= "1001111" ; a_to_i <= "1001100" ; a_to_j <= "1001111";
when "1110" => a_to_g <= "1001100" ; a_to_h <= "1001111" ; a_to_i <= "0100100" ; a_to_j <= "1001111";
when others => a_to_g <= "0100100" ; a_to_h <= "1001111" ; a_to_i <= "0100000" ; a_to_j <= "1001111";
```

```
process(clock_100Mhz,reset)
begin
	if reset = '1' then
		refresh <= (others => '0');
		tambah <= (others => '0');
	elsif rising_edge(clock_100Mhz)  then
		if refresh = "11000011010100000000" then
			refresh <= (others => '0');
		else
			refresh <= refresh + 1;
		end if;

		if tambah = "11" then
			tambah <= (others => '0');
		else
			tambah <= tambah + 1;
		end if;
	end if;
end process;

process(refresh,tambah)
begin
	case tambah is
	when "00" => oo <= a_to_g;
	when "01" => oo <= a_to_h;
	when "10" => oo <= a_to_i;
	when "11" => oo <= a_to_j;
	when others => oo <= (others => '0');
	end case;
end process;

end Behavioral;
```

c. Program miniproject.ucf (konfigurasi pin)

```
NET "oo(0)" LOC = N16;
NET "oo(1)" LOC = F13;
NET "oo(2)" LOC = R16;
NET "oo(3)" LOC = P15;
NET "oo(4)" LOC = N15;
NET "oo(5)" LOC = G13;
NET "oo(6)" LOC = E14;
NET "an(0)" LOC = D14;
NET "an(1)" LOC = G14;
NET "an(2)" LOC = F14;
NET "an(3)" LOC = E13;
NET "dp" LOC = P16;
NET "sw(0)" LOC = F12;
NET "sw(1)" LOC = G12;
NET "sw(2)" LOC = H14;
NET "sw(3)" LOC = H13;
```

d. Output Spartan 3

Analisa

Program VHDL ini memanfaatkan teknik multiplexing untuk secara bergantian menampilkan empat digit pada satu tampilan 7-segmen. Nilai biner dari switch dikonversi menjadi format heksadesimal dan ditampilkan pada dua digit pertama. Setelah itu, nilai tersebut ditambah 1, dikonversi kembali menjadi heksadesimal, dan ditampilkan pada dua digit terakhir. Pergantian antara digit-digit dikendalikan oleh pembagi clock ganda, yang memastikan transisi antar digit berlangsung lancar. Penggunaan switch memungkinkan fleksibilitas dalam menentukan nilai yang akan ditampilkan.

Kesimpulan

Dari percobaan yang telah dilakukan, terlihat bahwa implementasi tampilan 7-segmen menggunakan VHDL dan FPGA Xilinx Spartan 3 berhasil dilakukan, dengan fitur penting seperti clock divider. Pada tahap eksperimen berikutnya, ditambahkan fitur tambahan seperti penghilangan digit nol terdepan (leading zero blanking). Selain itu, nilai yang ditampilkan dapat dikendalikan melalui input switch dan tombol push button, memberikan fleksibilitas lebih. Pada tahap mini proyek, tampilan 7-segmen 4 digit digunakan untuk menampilkan pola-pola tertentu berdasarkan input dari switch, menggabungkan berbagai teknik yang telah dipelajari sebelumnya.